

Project management best practices

IST 400/600
Fall 2026
Collin Capano



Today

- Establishing best practices for project management with your mentor.
- Best practices for project management on your computer: folders and environments.
 - File system review
 - Environments

I. Establishing best practices with your mentor

Project planning

- Congratulations! You've gotten your project assignments
- Next establish a meeting time with your mentor.
 - Should plan to meet at least once a week.
 - Please meet with them sometime in the next week.
 - You can meet during the second half of class time (after 3:30)

Goals for your first meeting

In your first meeting, you need to:

1. Establish goals & a rough timeline to accomplish them (“scoping”)
2. Establish communication expectations
3. Establish how you will track issues and progress

Setting Reasonable Milestones/Expectations



Set reasonable overall goals for the project, then break that work into small, demonstrable increments. Aim for milestones that can be completed in 1–2 weeks. Each milestone should produce something tangible (e.g., a working feature, a passing test suite).



Estimate conservatively, then add buffer. Research software often involves exploration and uncertainty. A good rule of thumb: make your best estimate, then multiply by 2x.



Account for the “hidden” work. Documentation, testing, and deployment are key parts of the work, especially for open-source software. Build time for them into your milestones rather than treating them as afterthoughts.

- Schedules and goals frequently change as you work. That's ok! Unexpected things arise. Just keep a note of why when they. You should be able to explain why.
- **Write down your plan for the semester and email it to me.** Can be Word/Google doc, markdown, or text file

Setting communication expectations



You will need to communicate with your mentor outside of weekly meeting times.



Two types of discussions: technical decisions and quick questions.



Establish how type each will be discussed



Technical decisions: GitHub Issues? Pull requests?



Quick questions: Slack? Teams?
Email?

If you and your advisor would like a Slack account, I can add you to the OSPO slack. Just let me know.

Communicate regularly!

- A common beginner mistake is to wait until a weekly meeting to discuss a delay or problem. Don't do that!
- **Be proactive about discussing problems!**
- A quick Slack message like "blocked on X" or "need help with Y" keeps small issues from becoming big delays.

Establish how to track progress

- You will need to have a means to track progress
- Again, two levels of tracking: formal and informal
- Formal tracking – code version control, monitoring milestones
- Use GitHub for version control
- Recommend using GitHub Issues/Projects for monitoring milestones. Your mentor may use other tools (e.g., Jira)

Informal tracking

- You will want a place to quickly & easily jot down notes
- Also keep track of meeting minutes!
- I recommend creating a shared document for this
 - Can be a Google doc or Word doc
 - Your mentor may have a preferred method
- You and your mentor should both have access to it

Summary

By next
week you
should have
established:

A regular meeting time with your mentor.

Goals and milestones, written down.

How to communicate outside of meeting time.

How to track project milestones.

How to keep meeting minutes and quick notes.

Resources

- For more on best practices for managing your project see:

<https://opensource.syracuse.edu/best-practices/>



II. Managing your project on your computer

Where to store stuff on your computer

- Being organized about where and how you store files on your computer will make for a successful project.
- This means:
 - Keep a clean file structure on your computer for storing project information.
 - Use environments to manage installed software related to the project.

Your username is "aturing". Where is the home directory on your computer?

Mac users

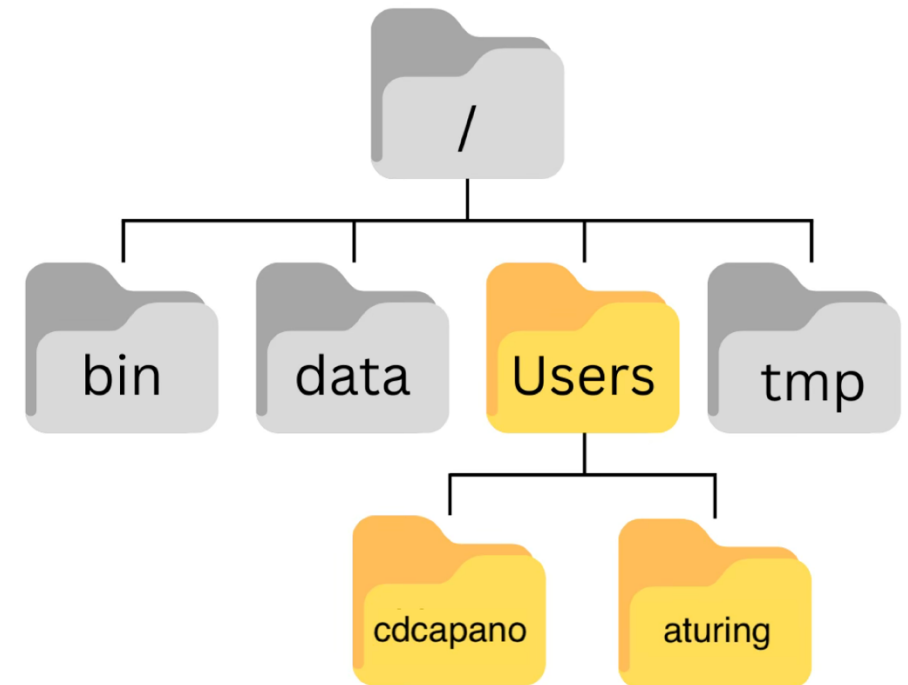
- A. /home/aturing
- B. /Users/aturing**
- C. /usr/local/aturing
- D. I don't know

Windows users

- A. C:\home\aturing
- B. C:\Users\aturing**
- C. C:\Documents\aturing
- D. I don't know

Files on your computer

- Your operating system provides means for you to view, manipulate, and execute files on your computer
- To view the collection of files on your computer (the file system):
 - On Windows: File Explorer
 - On Mac: Finder
- Files are organized hierarchically

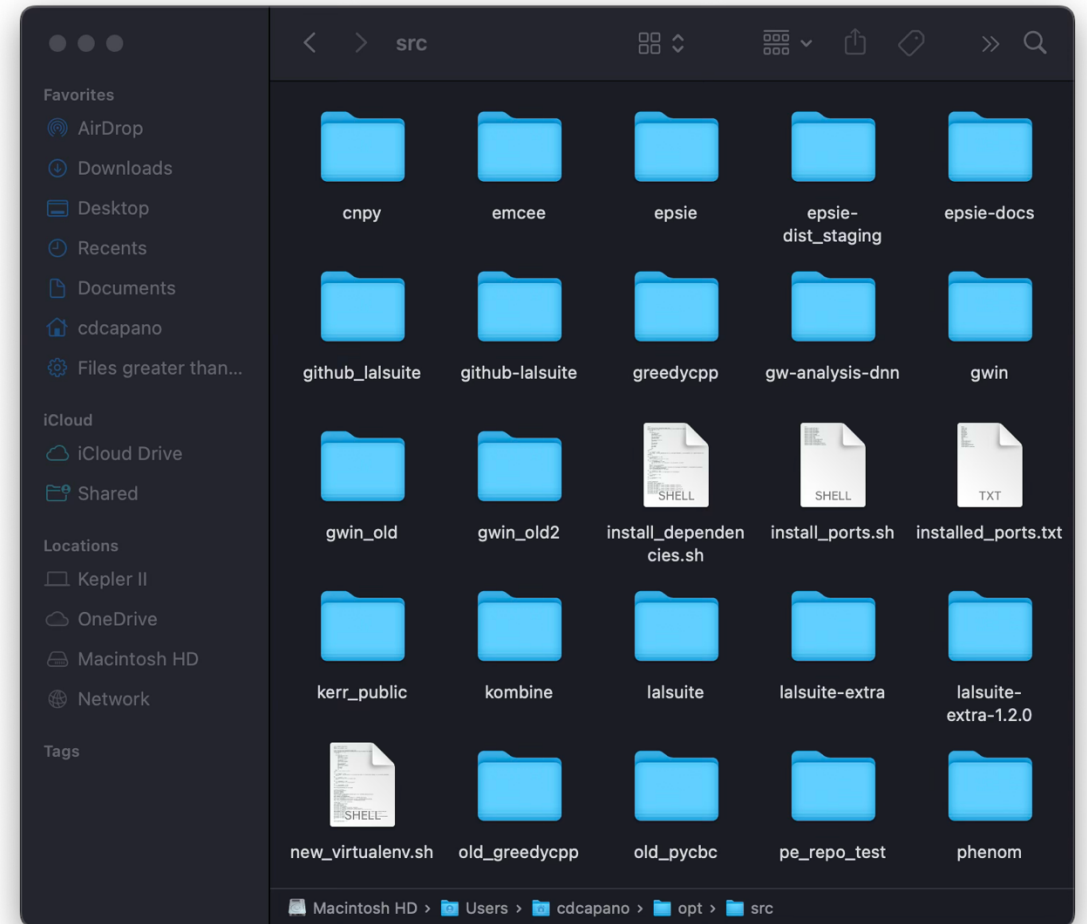


Changing the file system

- You can create folders!
- Using Finder (Mac) / Explorer (Windows) you can create new folders within folders to store stuff.
- Also possible to do with the command line
 - On both Mac/Linux (using Terminal) and Windows (using Powershell): `mkdir`

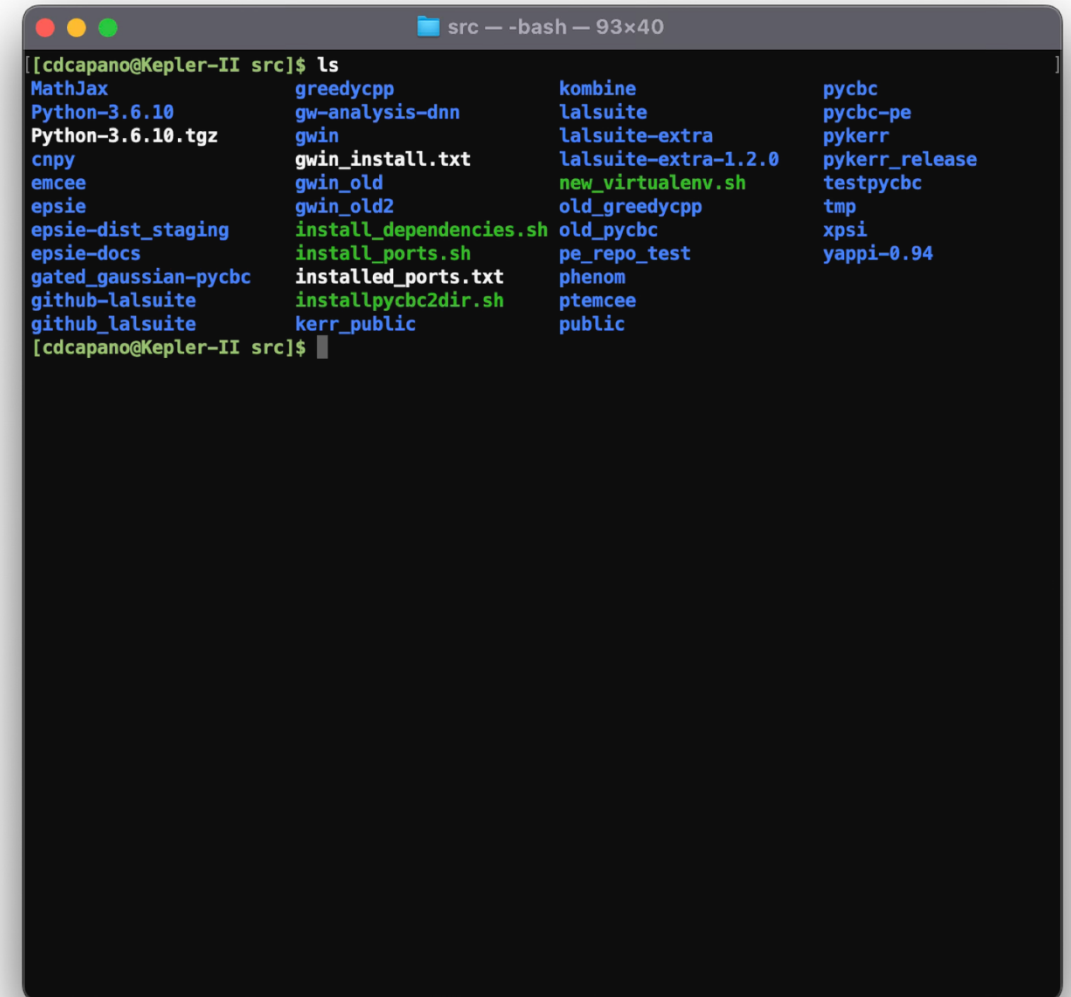
Types of Operating Systems

- 3 primary operating systems exist on the market today:
 - Windows
 - MacOS
 - Linux (several different flavors; Ubuntu is the most popular)
- All of these use **Graphical User Interfaces (GUIs)**
- GUIs are (mostly) intuitive; make it easy to accomplish tasks on a computer



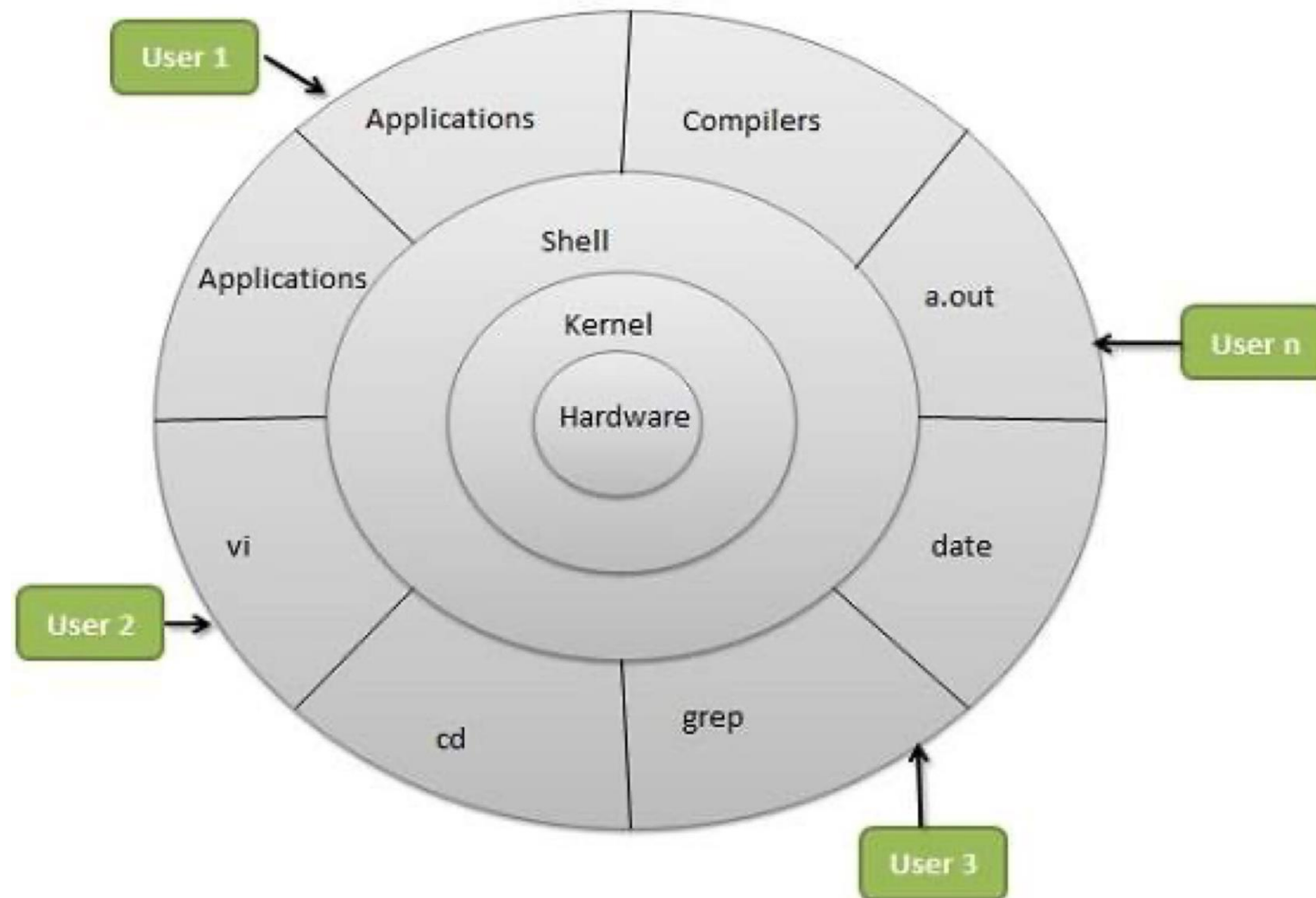
Command line

- All of these operating systems also provide a command line interface (CLI)
- The CLI consists of a text box in which you type commands to interact with the computer
- The program you use to do this is known as a terminal or shell
- Many different types of CLI shells; most popular in UNIX is bash shell



```
src — -bash — 93x40
[[cdcapano@Kepler-II src]$ ls
MathJax          greedyccp        combine           pycbc
Python-3.6.10    gw-analysis-dnn  lalsuite         pycbc-pe
Python-3.6.10.tgz gwin             lalsuite-extra   pykerr
cnp              gwin_install.txt lalsuite-extra-1.2.0 pykerr_release
emcee            gwin_old         new_virtualenv.sh testpycbc
epsie            gwin_old2        old_greedyccp    tmp
epsie-dist_staging install_dependencies.sh old_pycbc        xpsi
epsie-docs       install_ports.sh  pe_repo_test     yappi-0.94
gated_gaussian-pycbc installed_ports.txt phenom
github-lalsuite  installpycbc2dir.sh ptemcee
github_lalsuite  kerr_public      public
[[cdcapano@Kepler-II src]$
```

Anatomy of an OS



CLI vs GUI

- It's up to you whether to use the CLI
- I use a mixture of both (it's like being bilingual)
- Use whatever you feel most comfortable with...
- But don't be afraid to try the CLI!

Creating a folder for your project

- You should create a folder for storing all of your files related to this project
 - This will include any git repositories you clone
- I recommend creating a projects folder in your home directory, then creating a folder in there for this project; e.g.:

`/Users/{username}/projects/{project_name}`

- Tip: Don't use spaces in folder and file names!

Exercise

1. Create a folder in your home directory called projects. You can use either the GUI or the CLI to do this.
2. In projects, create another folder for your project. Give it a short, unique name with no spaces.
3. Create a new Word document. Save it to your project directory. Name the document "test.docx"
4. Close the document. Now open it again.
5. Open the folder in VS Code.

Environments

Managing installed code

- For most projects you will need to install software.
- This can include dependencies for the project, as well as the project itself.
- Frequently, you will work on multiple projects at once, or multiple versions of that project.
- Can be difficult to manage this by hand.

Package managers

- Enter: package managers!
- These are utilities that will download and install software for you, making sure to properly handle all dependencies in the process
- Some will also manage environments for you so you don't get version conflicts

Examples:

- Ubuntu: apt-get
 - >> apt-get install python3
- MacOS: brew or macports:
 - >> port install python3

conda

- System installers like apt-get install packages system-wide
- Can be problematic if you need to use multiple versions of the same code
- conda is a package manager that will install code into an **environment**
- Example:
 - >> conda create -n myenv
 - >> conda activate myenv
 - >> conda install python=3.8

Environments

- An environment is a collection of variable names the OS uses to determine where to load programs from and manage other custom settings.
- Environment variables store settings that affect the behavior of the shell and programs.
- Examples include:
 - PATH — where to search for executables
 - HOME — your home directory
 - USER — your username

Use an environment!

- You should create an environment for installing codes related to your project
- I recommend using conda, but if you are more familiar with another manager (e.g., uv) use that
 - I will post instructions to the course website on how to install conda and use it

Questions?

